32-Bit Ripple Carry Adder: DFT Insertion and ATPG

VLSI Testing and Verification (EC-513)

Faculty: Dr. P. Ranga Babu

Submitted by:

Pranjal Upadhyay

Roll No: **523EC0012**

Integrated B.Tech and M.Tech

Department of Electronics and Communication Engineering

Indian Institute of Information Technology Design and Manufacturing, Kurnool

April 16, 2026

0 Contents

1 Abstract **3**

2 Introduction **3**

3 Design Architecture **3**
- 3.1 Ripple Carry Adder – Theory . . . 3
- 3.2 Design Hierarchy . . . 4
- 3.3 Port List . . . 4

4 RTL Source Code **4**
- 4.1 Full Adder – `full_adder.v` . . . 4
- 4.2 Top-Level RCA – `rca_32bit.v` . . . 5

5 DFT Insertion Flow – Cadence Genus **6**
- 5.1 Synthesis and DFT TCL Script . . . 6
- 5.2 SDC Constraints – `rca_32bit.sdc` . . . 7

6 Synthesis Results **8**
- 6.1 Design Identity . . . 8
- 6.2 Timing Summary . . . 8
- 6.3 Area Summary . . . 8
- 6.4 Gate-Level Cell Breakdown . . . 9
- 6.5 Power Summary . . . 9

7 DFT Results **9**
- 7.1 DFT Configuration . . . 9
- 7.2 DFT Rule Check Results . . . 10
- 7.3 Scan Chain Configuration . . . 10
- 7.4 Scan Chain Register Order . . . 10
- 7.5 Output Files Generated by Genus . . . 11

8 ATPG Flow – Cadence Modus **11**
- 8.1 Modus ATPG Script . . . 11
- 8.2 Design Model Statistics . . . 12

9 ATPG Results **12**
- 9.1 Fault Model Statistics . . . 12
- 9.2 Scan Chain Test Coverage . . . 13
- 9.3 Logic Test Generation – Phase by Phase . . . 13
- 9.4 Final Fault Coverage . . . 13
- 9.5 Final Test Pattern Statistics . . . 13
- 9.6 Scan Chain Structural Verification . . . 13
- 9.7 DFT Signal Pin Assignment . . . 14

10 Discussion and Analysis **14**
10.1 Why Dynamic Fault Coverage is 42.93% 14
10.2 Stuck-At Coverage at 100% 14
10.3 Scan Mux Area Overhead 15
10.4 Comparison: RCA vs. Kogge-Stone Adder 15

11 Complete Results Summary **16**

12 Conclusion **16**

References **17**

1 Abstract

This report documents the complete Design for Testability (DFT) insertion and Automatic Test Pattern Generation (ATPG) flow for a 32-bit registered Ripple Carry Adder (RCA) implemented in 180 nm CMOS technology. The design is synthesized using Cadence Genus (v20.11) with muxed-scan DFT insertion, and subsequently subjected to ATPG using Cadence Modus (v20.12) in FULLSCAN mode.

The synthesized netlist contains 282 standard cells mapping to a total area of 3233.477 μm^2 and a post-DFT critical path slack of +12605 ps, confirming correct timing closure. A single scan chain of 98 flip-flops is stitched from `scan_in` to `scan_out` with zero DFT rule violations. ATPG achieves **100% stuck-at fault coverage** (3762/3762 faults detected) using 19 test patterns. Scan chain integrity is verified by Modus `verify_test_structures` with no structural violations detected.

2 Introduction

Modern VLSI designs are manufactured at nanometre scales where process variation, gate oxide defects, and interconnect shorts create faults that differ from the intended logic. Design for Testability (DFT) is the discipline of embedding controllability and observability structures into a design so that manufacturing faults can be detected by applying test patterns. Scan-based DFT is the dominant industrial methodology, converting flip-flops into scan cells and connecting them into one or more shift-register chains.

Automatic Test Pattern Generation (ATPG) tools exploit the scan infrastructure to generate compact test pattern sets that exercise each potential fault site. The Stuck-At Fault (SAF) model is the most widely used, assuming that a node is permanently stuck at logic 0 or logic 1 independent of its intended value.

This assignment applies the complete DFT+ATPG flow to a 32-bit Ripple Carry Adder:

1. RTL design in Verilog (two-file hierarchy: `full_adder.v` and `rca_32bit.v`).
2. Logic synthesis and muxed-scan DFT insertion using Cadence Genus.
3. ATPG and fault simulation using Cadence Modus in FULLSCAN mode.
4. Analysis of fault coverage, scan chain structure, and DFT rule compliance.

3 Design Architecture

3.1 Ripple Carry Adder – Theory

A Ripple Carry Adder (RCA) computes the sum of two n-bit operands by cascading n full-adder (FA) cells. Each FA computes:

$$S_i = A_i \oplus B_i \oplus C_i \tag{1}$$

$$C_{i+1} = A_i B_i + C_i(A_i \oplus B_i) \tag{2}$$

where C_0 is the carry-in. Because the carry propagates serially from bit 0 to bit 31, the worst-case delay is $O(n)$. For a 32-bit adder this gives:

$$t_{RCA} = t_{FA_carry} \times 32$$

The RCA is the simplest and most area-efficient adder architecture, making it suitable for low-speed, area-constrained applications. In this assignment it is used as the DUT for the DFT and ATPG flow.

3.2 Design Hierarchy

The RTL is organized as a two-level hierarchy:

Module	File	Description
`rca_32bit`	`rca_32bit.v`	Top-level sequential wrapper
`full_adder`	`full_adder.v`	Combinational 1-bit FA cell

The top module registers all inputs (`a_in`, `b_in`, `cin_in`) and outputs (`sum_out`, `cout_out`) using flip-flops clocked by `clk`. A `generate` loop instantiates 32 `full_adder` cells. DFT ports (`scan_en`, `scan_in`, `scan_out`) are included at RTL level.

3.3 Port List

Port	Direction	Width	Description
`clk`	Input	1	System clock
`reset`	Input	1	Active-high async reset
`scan_en`	Input	1	DFT: scan shift enable (active high)
`scan_in`	Input	1	DFT: scan chain input
`test_mode`	Input	1	DFT: test mode select
`scan_out`	Output	1	DFT: scan chain output
`a_in`	Input	32	Operand A
`b_in`	Input	32	Operand B
`cin_in`	Input	1	Carry-in
`sum_out`	Output	32	Registered 32-bit sum
`cout_out`	Output	1	Registered carry-out

4 RTL Source Code

4.1 Full Adder – `full_adder.v`

```
`timescale 1ns / 1ps

module full_adder(
    input  wire a,
    input  wire b,
    input  wire cin,
    output wire sum,
    output wire cout
);
    // Standard Boolean logic for sum and carry-out
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));

endmodule
```

Listing 1: full_adder.v – 1-bit combinational full adder

4.2 Top-Level RCA – rca_32bit.v

```
`timescale 1ns / 1ps

module rca_32bit (
    input  wire        clk,
    input  wire        reset,
    // --- DFT Ports ---
    input  wire        scan_en,
    input  wire        scan_in,
    input  wire        test_mode,
    output wire        scan_out,
    // -----------------
    input  wire [31:0] a_in,
    input  wire [31:0] b_in,
    input  wire        cin_in,
    output reg  [31:0] sum_out,
    output reg         cout_out
);
    // Internal registers to hold inputs
    reg  [31:0] a_reg, b_reg;
    reg         cin_reg;

    // Wires to capture combinational output from the RCA
    wire [31:0] sum_w;
    wire        cout_w;

    // Internal carry chain
    wire [32:0] carry;
    assign carry[0] = cin_reg;
    assign cout_w   = carry[32];

    // Instantiate 32 Full Adders
    genvar i;
    generate
        for (i = 0; i < 32; i = i + 1) begin : FA_LOOP
            full_adder fa_inst (
                .a    (a_reg[i]),
                .b    (b_reg[i]),
                .cin  (carry[i]),
                .sum  (sum_w[i]),
                .cout (carry[i+1])
            );
        end
    endgenerate

    // Sequential Logic: Clocking data in and out
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            a_reg    <= 32'd0;
            b_reg    <= 32'd0;
            cin_reg  <= 1'b0;
            sum_out  <= 32'd0;
```

```
            cout_out <= 1'b0;
        end else begin
            a_reg    <= a_in;
            b_reg    <= b_in;
            cin_reg  <= cin_in;
            sum_out  <= sum_w;
            cout_out <= cout_w;
        end
    end

    // Tie off scan_out for RTL simulation;
    // Genus overrides this during DFT insertion
    assign scan_out = 1'b0;

endmodule
```

Listing 2: rca_32bit.v – 32-bit registered RCA with DFT ports

5 DFT Insertion Flow – Cadence Genus

5.1 Synthesis and DFT TCL Script

The Genus script `run_genus_dft.tcl` automates the complete synthesis and DFT flow in a single run. The key steps are:

1. Load the 180 nm standard cell library (`slow.lib`).
2. Read RTL (`full_adder.v`, `rca_32bit.v`).
3. Elaborate and apply SDC timing constraints.
4. Set `dft_scan_style` to `muxed_scan`.
5. Run `syn_generic`, `syn_map`, and `syn_opt`.
6. Execute `check_dft_rules`, `replace_scan`, and `connect_scan_chains`.
7. Run post-DFT incremental optimization (`syn_opt -incr`).
8. Write reports and Modus ATPG files.

```
# Library and RTL setup
set_db init_lib_search_path /home/install/FOUNDRY/digital/180nm/dig/lib/
set_db library slow.lib
read_hdl ./full_adder.v
read_hdl ./rca_32bit.v
elaborate rca_32bit
read_sdc ./rca_32bit.sdc

# DFT configuration
set_db dft_scan_style muxed_scan
set_db dft_prefix     DFT_
define_dft shift_enable -name scan_en_sig -active high scan_en
define_dft test_clock   -name clk_test    -period 10000 clk

# Synthesis
set_db syn_generic_effort high;  syn_generic
set_db syn_map_effort high;      syn_map
set_db syn_opt_effort high;      syn_opt

# DFT insertion
```

```
check_dft_rules > ./dft_rules_check.rpt
replace_scan
define_scan_chain -name chain1 -sdi scan_in -sdo scan_out -
    non_shared_output
connect_scan_chains
syn_opt -incr

# Write outputs
write_hdl     > ./rca_32bit_post_dft.v
write_sdf     > ./rca_32bit_post_dft.sdf
write_sdc     > ./rca_32bit_post_dft.sdc
write_scandef > ./rca_32bit.scandef
write_dft_atpg -library ./rca_32bit_post_dft.v -directory ./
```

Listing 3: run_genus_dft.tcl – Genus synthesis and DFT script (key excerpts)

5.2 SDC Constraints – `rca_32bit.sdc`

```
# Clock: 50 MHz => period 20 ns
create_clock -name clk -period 20.0 [get_ports clk]
set_clock_uncertainty 0.5 [get_clocks clk]
set_clock_transition  0.2 [get_clocks clk]

# I/O delays (25% of clock period)
set_input_delay  5.0 -clock clk [get_ports {reset a_in b_in cin_in
    test_mode}]
set_output_delay 5.0 -clock clk [get_ports {sum_out cout_out}]

# DFT ports: false paths during functional analysis
set_false_path -from [get_ports scan_en]
set_false_path -from [get_ports scan_in]
set_false_path -to   [get_ports scan_out]

# Drive and load
set_driving_cell -lib_cell INVX1 -pin ZN \
    [get_ports {clk reset test_mode scan_en scan_in}]
set_load 0.05 [all_outputs]

# DRC constraints
set_max_transition 0.5 [current_design]
set_max_fanout     20  [current_design]
```

Listing 4: rca_32bit.sdc – Timing constraints for 180 nm, 50 MHz clock

6 Synthesis Results

6.1 Design Identity

Parameter	Value
Top module	`rca_32bit`
Technology	180 nm CMOS (slow.lib corner)
Tool	Cadence Genus Synthesis Solution v20.11-s111_1
Run date	April 16, 2026 – 13:19:48 IST
Operating condition	slow (balanced_tree)
Wireload mode	enclosed

6.2 Timing Summary

Parameter	Value
Clock name	`clk`
Clock period	20 ns (50 MHz)
Clock uncertainty	0.5 ns
Critical path start	`a_reg_reg[0]/CK`
Critical path end	`sum_out_reg[31]/D`
Data path delay	6673 ps
Setup time	222 ps
Required time	19278 ps
Slack (WNS)	**+12605 ps — MET**
TNS	0 ps (no violations)

The critical path traverses the ripple carry chain: from the output of `a_reg_reg[0]` through 32 alternating `AOI22XL`/`OAI21XL` carry cells to the data input of `sum_out_reg[31]`. The 12605 ps of positive slack means the design comfortably meets the 20 ns clock constraint and could operate at approximately 10 ns (100 MHz) with similar margin.

6.3 Area Summary

Category	Instances	Area (μm^2)	%
Sequential (SDFF*)	98	2448.571	75.7%
Logic (combinational)	112	605.520	18.7%
Inverters	65	147.596	4.6%
Buffers	7	31.790	1.0%
Total	**282**	**3233.477**	**100%**

6.4 Gate-Level Cell Breakdown

Cell type	Count	Area (μm^2)	Role
`SDFFRHQX1`	97	2422.837	Scan flip-flop (x1 drive)
`SDFFRHQX2`	1	25.735	Scan flip-flop (x2 drive)
`MXI2XL`	64	387.533	Scan mux (DFT overhead)
`INVXL`	65	147.596	Inverter
`AOI22XL`	16	96.883	AND-OR-Invert (carry)
`OAI21XL`	16	72.662	OR-AND-Invert (carry)
`NAND2XL`	16	48.442	NAND gate
`BUFX2`	7	31.790	Buffer
Total	**282**	**3233.477**	

Note: The 64 `MXI2XL` cells (387.533 μm^2, 12% of total area) are the scan multiplexer overhead introduced by muxed-scan DFT insertion. In functional mode, these muxes pass the combinational data; in scan mode, they select the scan-in data for shifting.

6.5 Power Summary

Category	Leakage (W)	Internal (W)	Switching (W)	Total (W)	%
Register	1.498e-05	1.183e-04	1.899e-05	1.523e-04	88.33%
Logic	3.074e-06	6.590e-06	3.517e-06	1.318e-05	7.65%
Clock	0	0	6.946e-06	6.946e-06	4.03%
Total	**1.805e-05**	**1.249e-04**	**2.945e-05**	**1.724e-04**	**100%**

Register cells dominate power at 88.33%, consistent with a design where 75.7% of area is sequential. Internal switching accounts for 72.45% of total power, leakage for 10.47%.

7 DFT Results

7.1 DFT Configuration

Parameter	Value
Scan style	`muxed_scan`
DFT prefix	`DFT_`
Shift enable port	`scan_en` (active high)
Scan data-in port	`scan_in`
Scan data-out port	`scan_out`
Test clock name	`clk_test`
Test clock source	`clk`
Test clock period	10000 ps (10 ns, 100 MHz)
Test clock edge	Rising
Lock-up element type	preferred_level_sensitive
Mix clock edges	false

7.2 DFT Rule Check Results

Genus `check_dft_rules` was run both before and after scan chain insertion. The results are identical and show:

Rule category	Violations	Status
Internally driven clock net	0	✓
Tied constant clock net	0	✓
Undriven clock net	0	✓
Conflicting async & clock net	0	✓
Misc clock net	0	✓
Internally driven async net	0	✓
Tied active async net	0	✓
Undriven async net	0	✓
Misc async net	0	✓
Total DFT violations	**0**	✓ **All pass**
Registers passing DFT rules	98 (100%)	✓
Registers failing DFT rules	0	✓

7.3 Scan Chain Configuration

Parameter	Value
Chain name	`chain1`
Scan-in	`scan_in`
Scan-out	`scan_out`
Shift enable	`scan_en` (active high)
Clock domain	`clk_test` (rising edge)
Chain length	98 flip-flops
Total design registers	98
Scannable percentage	**100%**

7.4 Scan Chain Register Order

All 98 flip-flops are stitched in the following order, forming a single serial shift register from `scan_in` to `scan_out`:

Bit positions	Registers	Count
1 – 32	`a_reg_reg[0]` → `a_reg_reg[31]`	32
33 – 64	`b_reg_reg[0]` → `b_reg_reg[31]`	32
65	`cin_reg_reg`	1
66	`cout_out_reg`	1
67 – 98	`sum_out_reg[0]` → `sum_out_reg[31]`	32
	Total	**98**

This ordering reflects the logical grouping: input operand registers first (A, B, Cin), then output registers (Cout, Sum), which is a natural ordering for scan shift.

7.5 Output Files Generated by Genus

File	Purpose
rca_32bit_post_dft.v	Scan-inserted gate-level netlist
rca_32bit_post_dft.sdc	Post-DFT timing constraints
rca_32bit_post_dft.sdf	Standard Delay Format for simulation
rca_32bit.scandef	Scan chain definition for Modus
rca_32bit.FULLSCAN.pinassign	ATPG pin assignment file
rca_32bit.test_netlist.v	Test-mode netlist
post_dft_timing.rpt	Post-DFT timing report
post_dft_area.rpt	Post-DFT area report
post_dft_power.rpt	Post-DFT power report
post_dft_gates.rpt	Gate count and type breakdown
scan_chains.rpt	Scan chain configuration
dft_rules_check.rpt	DFT rule checker output
dft_setup.rpt	DFT setup summary

8 ATPG Flow – Cadence Modus

8.1 Modus ATPG Script

```
set WORKDIR ./
set CELL     rca_32bit
set NETLIST  $WORKDIR/rca_32bit_post_dft.v
set LIBRARY  "/home/install/FOUNDRY/digital/180nm/dig/vlog/typical.v"
set TESTMODE FULLSCAN
set RESULTS_DIR $WORKDIR/results
file mkdir $RESULTS_DIR

# Locate pin assignment and mode definition files from Genus
catch { set ASSIGNFILE [exec bash -c \
    "find $WORKDIR -type f -name \"*.pinassign\" | head -n 1"] }

# Build model
build_model -cell $CELL -workdir $WORKDIR \
    -designsource $NETLIST -techlib $LIBRARY \
    -designtop $CELL -allowmissingmodules yes

# Build FULLSCAN test mode
build_testmode -workdir $WORKDIR -testmode $TESTMODE \
    -assignfile $ASSIGNFILE

# Verify and report scan chain structure
verify_test_structures -workdir $WORKDIR -testmode $TESTMODE \
    > $RESULTS_DIR/verify_structures.rpt
report_test_structures -workdir $WORKDIR -testmode $TESTMODE \
    > $RESULTS_DIR/test_structures.rpt

# Build stuck-at fault model
build_faultmodel -fullfault yes

# Generate ATPG patterns (scan + logic)
create_scanchain_tests -testmode $TESTMODE -experiment scan
```

```
create_logic_tests     -testmode $TESTMODE -experiment logic

# Report coverage and write vectors
redirect $RESULTS_DIR/test_coverage_logic.rpt {
    report_statistics -experiment logic
}
write_vectors -testmode $TESTMODE -inexperiment logic \
    -language verilog -scanformat serial \
    -outputfilename $RESULTS_DIR/test_results.v
```

Listing 5: run_modus_atpg.tcl – Modus ATPG script (key excerpts)

8.2 Design Model Statistics

After `build_model`, Modus reports the following design structure:

Parameter	Value	Notes
Hierarchical blocks	3044	
Flattened nodes	2212	
Total pins	9608	
Total nets	5380	
Primary inputs	70	
Primary outputs	34	
Total flops	98	All rising-edge, set-dominant reset
MUX2 primitives	162	Scan muxes from DFT insertion
Tied-to-0 nets	196	
Tied-to-1 nets	98	
Tech library cells	283	
Test mode	FULLSCAN (GSD scan type)	

9 ATPG Results

9.1 Fault Model Statistics

Fault class	Total	Active	Inactive
Total static (stuck-at)	3762	3762 (100%)	0
Collapsed static	2852	2852	0
PI static	138	138	0
PO static	68	68	0
Total dynamic (transition)	3818	3210 (84.1%)	608
Collapsed dynamic	2488	2076 (83.4%)	412
IDDq faults	3762	3762	0
Global ignored static	198	—	—
Global ignored dynamic	394	—	—

Observation: All 3762 static stuck-at faults are active (observable) in FULLSCAN mode. Of the 3818 dynamic (transition delay) faults, only 84.1% are active; the remaining 608 are inactive because the carry chain topology makes certain transition faults unobservable without at-speed two-cycle patterns.

9.2 Scan Chain Test Coverage

`create_scanchain_tests` generates a single scan-load/unload sequence and measures coverage before logic ATPG:

Fault type	Total	Tested	Untested	%TCov	%ATCov
Static (stuck-at)	3762	2046	1716	54.39%	54.39%
Dynamic (transition)	3210	1378	1832	42.93%	42.93%

The single scan sequence detects 54% of stuck-at faults – a useful baseline before dedicated logic ATPG patterns are applied.

9.3 Logic Test Generation – Phase by Phase

`create_logic_tests` runs in three sequential phases:

Phase	Patterns (sim/eff)	New faults detected	ATCov after phase
Scan test	1 / 1	2042	54.28%
Reset/Set test	1 / 1	198	59.54%
Static logic	16–19 / 16–17	1520–1522	99.95%–99.99%
Final	**19 total**	**3762**	**100.00%**

9.4 Final Fault Coverage

Metric	Faults	Tested	Possibly	Redundant	Untested	%ATCov
Static (stuck-at)	**3762**	**3762**	**0**	**0**	**0**	**100.00%**
Dynamic (transition)	3210	1378	0	0	1832	42.93%

Key result: 100% of 3762 stuck-at faults are detected with 19 test patterns. The dynamic fault coverage of 42.93% is structurally limited by the ripple carry topology and the absence of at-speed two-pattern testing, which is expected for this fault class.

9.5 Final Test Pattern Statistics

Test section	Sequences
Scan chain test	1
Logic (ATPG)	18
Total	**19**

9.6 Scan Chain Structural Verification

Modus `verify_test_structures` checks all DFT structural properties:

Structural check	Status
Scan chain controllable (from SI)	✓
Scan chain observable (to SO)	✓
Longest chain = 98 bits (100% of avg)	✓
Control chains (SI-fed)	1
Observe chains (SO-fed)	1
Clock chopper primitives	0
Feedback loops / keeper devices	0
TSD contention	0
Clock signal races	0
Latch-related issues	0

9.7 DFT Signal Pin Assignment

Pin index	Type	DFT function	Port	Active
65	PI	−EC / −SC (edge + system clock)	`clk`	–
66	PI	−SC (async reset)	`reset`	Low
67	PI	+SE (scan enable)	`scan_en`	High
68	PI	SI (scan data input)	`scan_in`	–
71	PO	SO (scan data output)	`scan_out`	–

10 Discussion and Analysis

10.1 Why Dynamic Fault Coverage is 42.93%

Transition delay (dynamic) faults require a two-pattern test: a "launch" cycle sets a node to the wrong state, and a "capture" cycle observes whether the transition completes within the clock period. In FULLSCAN mode without at-speed capture, Modus uses a functional capture cycle, which cannot sensitize all internal carry transitions in the RCA chain. Additionally, 608 dynamic faults are classified inactive (unobservable) because:

1. The long carry chain makes intermediate nodes structurally unobservable without propagating through all downstream cells.
2. The registered output adds one additional clock cycle of latency that cannot be exploited in a single-capture pattern.

This is not a design error but a known characteristic of carry-chain adder architectures under static ATPG.

10.2 Stuck-At Coverage at 100%

The 100% stuck-at coverage is achievable because:

1. All 98 flip-flops are in the scan chain (full scan), giving direct controllability of every registered node.
2. The scan-out port provides direct observability of every flip-flop state after capture.
3. The combinational logic between registers is simple (FA cells), and 19 patterns are sufficient to sensitize every stuck-at fault site.

10.3 Scan Mux Area Overhead

The muxed-scan style inserts one 2:1 multiplexer per flip-flop to select between the functional data input and the scan data input. With 98 flip-flops and one additional mux per cell, the 64 `MXI2XL` cells visible in the gate report represent the scan mux overhead (64 because some FFs share mux resources after optimization). This 387.533 μm^2 overhead is 12% of total area – a typical and acceptable cost for full-scan testability.

10.4 Comparison: RCA vs. Kogge-Stone Adder

For context, Table 1 compares key metrics between this 32-bit RCA (DFT assignment) and the 32-bit Kogge-Stone Adder (prior work).

Table 1: Metric comparison: 32-bit RCA (DFT) vs. 32-bit KSA (RTL-to-GDSII)

Metric	**RCA (this work)**	**KSA (prior)**
Technology	180 nm	180 nm
Architecture	Ripple carry	Kogge-Stone parallel prefix
Critical path delay	6673 ps	4186 ps (pre-layout)
Area (post-synthesis)	3233 μm^2	2448 μm^2
Max frequency (approx.)	∼50 MHz	∼150 MHz
Scan chain length	98	N/A (no DFT)
Stuck-at fault coverage	**100%**	N/A
DFT rule violations	0	N/A

The KSA is significantly faster due to logarithmic carry computation, but the RCA's simpler structure enables a cleaner DFT insertion with 100% scan coverage.

11 Complete Results Summary

Metric	Value	Status
Synthesis		
Technology node	180 nm slow.lib	–
Total cells	282	–
Total area	3233.477 μm^2	–
Critical path delay	6673 ps	–
Slack (WNS)	+12605 ps	✓ MET
Total power	172.4 µW	–
DFT (Genus)		
Scan style	muxed_scan	–
Total flip-flops	98	–
Scan chain length	98	–
Scannable percentage	100%	✓
DFT rule violations	0	✓
Scan mux overhead (area)	387.5 μm^2 (12%)	–
ATPG (Modus, FULLSCAN)		
Total stuck-at faults	3762	–
Detected stuck-at faults	3762	–
Stuck-at coverage	**100.00%**	✓
Total dynamic faults	3818	–
Dynamic coverage	42.93%	Expected
Total test patterns	19 (1 scan + 18 logic)	–
Scan chain: controllable	Yes	✓
Scan chain: observable	Yes	✓
Structural violations	0	✓

12 Conclusion

This assignment successfully demonstrates the complete DFT and ATPG flow for a 32-bit Ripple Carry Adder in 180 nm CMOS:

1. **DFT insertion:** Cadence Genus inserted muxed-scan flip-flops and stitched a single 98-element scan chain from `scan_in` to `scan_out` with zero DFT rule violations and 100% register scanability.

2. **Timing:** Post-DFT synthesis meets the 20 ns clock constraint with a large positive slack of +12605 ps, indicating ample timing margin. The scan mux area overhead is 12% of total area.

3. **ATPG:** Cadence Modus achieves **100% stuck-at fault coverage** using only 19 test patterns. The dynamic fault coverage of 42.93% is structurally limited by the RCA carry chain topology and is consistent with expected behaviour for this fault class without at-speed two-pattern testing.

4. **Verification:** Modus `verify_test_structures` confirms the scan chain is fully controllable and observable with no structural defects.

The results validate the design's manufacturing testability and confirm that the DFT flow is correctly implemented from RTL through ATPG vector generation.

12 References

1. Cadence Design Systems, *Genus Synthesis Solution User Guide*, Version 20.11, 2021.
2. Cadence Design Systems, *Modus DFT Software Solution User Guide*, Version 20.12, 2021.
3. M. L. Bushnell and V. D. Agrawal, *Essentials of Electronic Testing for Digital, Memory and Mixed-Signal VLSI Circuits*, Springer, 2000.
4. V. D. Agrawal and S. C. Seth, *Tutorial: Automatic Test Pattern Generation*, IEEE Design and Test of Computers, 1988.
5. N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Addison-Wesley, 2010.